

# The Complementation Problem for Büchi Automata with Applications to Temporal Logic (extended abstract)

*A. Prasad Sistla*

University of Massachusetts

*Moshe Y. Vardi*

Stanford University

*Pierre Wolper*

AT&T Bell Laboratories

## ABSTRACT

The problem of complementing Büchi automata arises when developing decision procedures for temporal logics of programs. Unfortunately, previously known constructions for complementing Büchi automata involve a doubly exponential blow-up in the size of the automaton. We present a construction that involves only an exponential blow-up. We use this construction to prove a polynomial space upper bound for the propositional temporal logic of regular events and to prove a complexity hierarchy result for quantified propositional temporal logic.

## 1. Introduction

For many years, logics of programs were tools for reasoning about the input/output behavior of programs. When dealing with concurrent or non-terminating processes (like operating systems) there is, however, a need to reason about infinite computation paths. These are the sequences of states that the computation goes through. In the propositional case they can be viewed as infinite sequences of propositional truth assignments. In [Pn81], temporal logic was proposed to reason about such sequences. Later it was incorporated into the process logics of [Ni80], [HKP82], and [VW83].

Recent works [Sis83, WVS83] established a close relationship between temporal logic and the theory of  $\omega$ -regular languages. The  $\omega$ -regular languages are the analogue of the regular languages, but defined on denumerable words rather than finite words. The notion of  $\omega$ -regularity is robust and has a well-developed theory [Ch74, Ei74, TB73]. There are several characterizations of  $\omega$ -regular languages, one of which is by Büchi automata [Bu62]. A Büchi automaton is a finite automaton operating on denumerable words. An infinite word is accepted by a Büchi automaton iff there is some run of the automaton on that word in which some state from a designated set of states appears infinitely often.

In [Sis83, WVS83] several temporal logics are shown to have exactly the expressive power of Büchi automata; in other words, the class of sets of sequences described by those logics coincides with the class of  $\omega$ -regular languages. One method to decide satisfiability for these logics is to build a Büchi automaton that accepts exactly the strings satisfying the formula. Since these logics are closed under negation, building this automaton involves complementing Büchi automata. That is, given a Büchi automaton  $A$ , one has to find a Büchi automaton  $\bar{A}$  such that  $L(\bar{A}) = \Sigma^* - L(A)$ , where  $L(A)$  denotes the language accepted by  $A$ .

The complementation problem for Büchi automata was first studied by Büchi [Bu62]. He showed that his automata are indeed closed under complementation. His proof, however, was not explicitly constructive. Later on, several explicit constructions were given [Bu73, Ch74, McN66, Sie70]. All these constructions, however, involve at least a doubly exponential blow-up. That is, there is a constant  $c > 1$  such that if  $A$  has  $n$  states, then  $\bar{A}$  has  $c^{c^n}$  states. This blow-up is very expensive computationally and causes the decision procedures using the complementation of Büchi automata to be highly inefficient. For example, the decision procedure described in [WVS83] for the temporal logic  $ETL$ , runs in exponential space, while the known lower bound for this logic is PSPACE.

In this paper we reexamine the complementation problem for Büchi automata. We prove, using Büchi's original ideas [Bu62], that Büchi automata can be complemented with only an exponential blow-up. We then use the construction to show that the non-emptiness of complement problem for Büchi automata, i.e., the problem of whether for a given Büchi automaton  $A$  we have  $L(\bar{A}) \neq \emptyset$ , is PSPACE-complete. (The analogous result for finite automata on finite words was proven by Meyer and Stockmeyer [MS72]).

These results turn out to be very useful in deciding satisfiability for various temporal logics. We first reconsider  $ETL$ , an extended temporal logic that directly reasons about  $\omega$ -regular events [WVS83]. As mentioned earlier, the best known decision procedure for this logic runs in exponential space. Using our results about Büchi automata, we improve the upper bound for  $ETL$  to polynomial space, which matches the lower bound.

We then turn to  $QPTL$ , a quantified propositional temporal logic [Sis83]. While this logic has the same expressive power as  $ETL$ , its complexity is nonelementary, since  $S1S$ , the second-order theory of one successor, which is known to be nonelementary [Me75], is easily reducible to  $QPTL$ . Using our result about Büchi automata we prove that the class of  $QPTL$  formulas with  $k$  alternations of quantifiers, is complete for  $SPACE(exp^k(p(n)))$  (i.e., a stack of  $k$  exponentials), where  $p(n)$  is a polynomial in  $n$ . We believe that this result is of general theoretical interest, since  $QPTL$  is the first nonelementary logic we know of, where each alternation of quantifiers increases the space complexity by exactly one exponential.

## 2. Büchi Automata and their Complementation Problem

A Büchi automaton is a nondeterministic finite automaton on denumerable words. Formally, it is a tuple  $A = (\Sigma, S, \rho, s_0, F)$ , where  $\Sigma$  is an alphabet,  $S$  is a set of states,  $\rho: S \times \Sigma \rightarrow 2^S$  is a nondeterministic transition function,  $s_0 \in S$  is a starting state, and  $F \subseteq S$  is a set of designated states. A run of  $A$  over a denumerable word  $w = a_1 a_2 \dots$ , is a sequence  $s_0, s_1, \dots$ , where  $s_i \in \rho(s_{i-1}, a_i)$ , for all  $i \geq 1$ . A run  $s_0, s_1, \dots$  is *accepting* if there is some designated state that repeats infinitely often, i.e., for some  $s \in F$  there are infinitely many  $i$ 's such that  $s_i = s$ . The denumerable word  $w$  is *accepted* by  $A$  if there is an accepting run of  $A$  over  $w$ . The set of denumerable words accepted by  $A$  is denoted  $L(A)$ .

We consider the problem of complementing Büchi automata. That is given a Büchi automaton  $A$  that accepts the language  $L(A)$ , we want to construct an automaton  $\bar{A}$  that accepts the language  $\bar{L}(A) = \Sigma^* - L(A)$ . This problem was first studied by Büchi [Bu62]. He showed that his automata are indeed closed under complementation. His proof, however, was not explicitly constructive. Later on, explicit versions of Büchi's original proof were given [Bu73, Sie70]. These explicit constructions, however, involve a doubly exponential blow-up. That is, there is a constant  $c > 1$  such that if  $A$  has  $n$  states, then  $\bar{A}$  has  $c^n$  states. Other construction for complementing Büchi automata were given by McNaughton [McN66] and by Rabin [Ra69] (see also [Ch74]). McNaughton's construction is also doubly exponential, while Rabin's construction is nonelementary.

Here we describe a construction, based on Büchi's original proof, that given a Büchi automaton  $A$  with  $n$  states, yields a Büchi automaton with  $O(16^n)$  states that accepts the complement of  $L(A)$ .

### 2.1. A Generalized Subset Construction

To complement a Büchi automaton  $A = (\Sigma, S, \rho, s_0, F)$ , we first build a family  $\{\bar{A}_i\}$  of deterministic automata on finite words that captures the essential behavior of the automaton. The behavior that we are trying to capture is as follows. Given a finite nonempty word  $x$  and two states  $u, v \in S$ :

1. is there a run of  $A$  on  $x$  starting with  $u$  and ending with  $v$ ?
2. is there a run of  $A$  on  $x$  starting with  $u$ , ending with  $v$ , and containing some state in  $F$ ?

To construct the automata  $\{\bar{A}_i\}$ , we use a construction that can be viewed as a generalization of Rabin and Scott's *subset construction* [RS59].

Let  $S = \{s_1, \dots, s_n\}$  be the set of states of  $A$ . Define  $S' = S \times \{0, 1\}$  and  $S^* = (2^{S'})^n$ .  $S^*$  has  $m$  states, denoted  $p_1, \dots, p_m$ , where  $m = 4^{n^2}$ . Intuitively, a state in  $S^*$  is an  $n$ -tuple of sets of states of  $S$  labeled by 0 or 1. We need an  $n$ -tuple of sets rather than a single set, because we are trying to capture information about runs that can start in any state of  $S$ . The label on the state (0 or 1) indicates whether the run contains a state in  $F$ . The state set of the  $\bar{A}_i$ 's is  $\bar{S} = S^* \cup \{p_0\}$ , i.e., we add to  $S^*$  a special starting state.

The deterministic transition function  $\bar{\rho}: \bar{S} \times \Sigma \rightarrow \bar{S}^*$  is defined as follows:

- $\langle S_1, \dots, S_n \rangle = \bar{\rho}(p_0, a)$  iff  $S_i = \{ \langle u, 0 \rangle : u \in \rho(s_i, a) \} \cup \{ \langle u, 1 \rangle : u \in \rho(s_i, a) \cap F \}$ .

- $\langle S_1, \dots, S_n \rangle = \bar{p}(\langle T_1, \dots, T_n \rangle, a)$  iff  
 $S_i = \{ \langle u, 0 \rangle : u \in \rho(v, a) \text{ for some } \langle v, j \rangle \in T_i \} \cup \{ \langle u, 1 \rangle : u \in \rho(v, a) \text{ for some } \langle v, 1 \rangle \in T_i \} \cup$   
 $\{ \langle u, 1 \rangle : u \in \rho(v, a) \cap F \text{ for some } \langle v, j \rangle \in T_i \}.$

We now define  $\tilde{A}_i$ ,  $1 \leq i \leq m$ , as the deterministic automaton on finite words  $(\Sigma, \tilde{S}, \bar{p}, p_0, \{p_i\})$ . Let  $X_i$  be the set of finite words accepted by  $\tilde{A}_i$ , i.e.,  $X_i = L(\tilde{A}_i)$ . The following lemma follows immediately from the fact that the  $\tilde{A}_i$ 's are deterministic.

**Lemma 2.1:**  $X_1, \dots, X_m$  is a partition of  $\Sigma^+$ .  $\square$

The next lemma describe how the  $\tilde{A}_i$ 's capture the behavior of  $A$ .

**Lemma 2.2:** Let  $p_i = \langle S_1, \dots, S_n \rangle$  and  $x \in \Sigma^+$ . Then  $x \in X_i$  iff for all pairs of states  $s_j, s_l$  of  $A$ :

1. there is a run  $u_0, \dots, u_k$ ,  $1 \leq k$ , of  $A$  over  $x$  such that  $u_0 = s_j$  and  $u_k = s_l$  iff  $\langle s_l, 0 \rangle \in S_j$ .
2. there is a run  $u_0, \dots, u_k$ ,  $1 \leq k$ , of  $A$  over  $x$  such that  $u_0 = s_j$ ,  $u_k = s_l$ , and  $u_k \in F$  for some  $1 \leq h \leq k$  iff  $\langle s_l, 1 \rangle \in S_j$ .  $\square$

In the next section we show how the generalized subset construction can be used to complement Büchi automata. We believe that this construction is of general usefulness. Two recent applications are the determinization of Büchi automata [Va85] and emptiness of hybrid automata [VS85].

## 2.2. The Complementation Construction

Consider now the languages  $Y_{ij} = X_i X_j^*$  where  $1 \leq i, j \leq m$ . We can prove the following results about these languages.

**Lemma 2.3:**  $\cup_{ij} Y_{ij} = \Sigma^*$

*Proof:* The proof is based on Ramsey's Theorem and is similar to the proof of Lemma 2.1 in [Bu62]. Ramsey's theorem tells us that given a partition of all pairs of natural numbers into finitely many disjoint sets  $C_1, \dots, C_n$ , there exists an infinite sequence of natural numbers  $i_1 < i_2 < \dots < i_q < \dots$  and a set  $C_k$  such that  $(i_p, i_q) \in C_k$  for all pairs  $i_p, i_q$  with  $i_p < i_q$ . Let us consider a denumerable word  $w = w_0 w_1 \dots$ . By Lemma 2.1, the word  $w$ , in combination with the languages  $X_i$  defined in the previous section, define a partition of all pairs of natural numbers into  $m$  sets,  $C_1, \dots, C_m$ , such that  $(i, j) \in C_k$  iff  $w_i \dots w_{j-1} \in X_k$ . By Ramsey's Theorem, there is a sequence  $i_1 < i_2 < \dots < i_q < \dots$  and a set  $C_k$  such that  $(i_p, i_q) \in C_k$  for all pairs  $i_p, i_q$  with  $i_p < i_q$ . This implies that the word  $x$  can be partitioned into  $w_1 = w_0 \dots w_{i_1-1}$ ,  $w_2 = w_{i_1} \dots w_{i_2-1}$ ,  $w_3 = w_{i_2} \dots w_{i_3-1}$ ,  $\dots$ , where for  $l > 1$ ,  $w_l \in X_j$  and  $w_1 \in X_i$  for some  $i$ . Thus, for some  $i, j$ ,  $x \in Y_{ij}$ .  $\square$

**Lemma 2.4:** For  $1 \leq i, j \leq m$ , either  $L(A) \cap Y_{ij} = \emptyset$  or  $L(A) \cap Y_{ij} = Y_{ij}$ .

*Proof:* We will prove that if one word  $w \in Y_{ij}$  is in  $L(A)$ , then all words in  $Y_{ij}$  are in  $L(A)$ . Indeed, a word  $w \in Y_{ij}$  can be decomposed into  $w_1 w_1^* w_2^* \dots$  where  $w_1 \in X_i$  and for all  $k > 0$ ,  $w_k^* \in X_j$ . Consider a run of  $A$  on  $w$  and denote by  $s_i$  the state reached in that run at the end of  $w_i$ ,  $s_j^1$  the state reached at the end of  $w_1 w_1^*$ , etc. The run is accepting iff the path taken through the automaton between  $s_i^k$  and  $s_j^{k+1}$  contains a state in  $F$  for infinitely many  $k$ . Now, any other word  $y \in Y_{ij}$  can be decomposed similarly to  $w$  into  $y_1 y_1^* y_2^* \dots$ . By Lemma 2.2, there will also be a run of  $A$  on  $y$  such that the state reached at the end of  $y_i$  will be  $s_i$ , the state reached at the end of  $y_j y_j^*$  will be  $s_j^1$ , etc. Moreover, there will be a path between  $s_i^k$  and  $s_j^{k+1}$  containing a state of  $F$  and labeled by  $y_j^k$  iff there is such a path labeled by  $x_j^k$ . Hence, if  $x$  is accepted by  $A$ , so is  $y$ .  $\square$

**Lemma 2.5:**  $\overline{L(A)} = \cup \{ Y_{ij} \mid Y_{ij} \cap L(A) = \emptyset \}.$

*Proof:* Immediate from Lemmas 2.3 and 2.4.  $\square$

We now construct a Büchi automaton  $\bar{A}$  that accepts  $\overline{L(A)}$ .

**Theorem 2.6:** Let  $A = (\Sigma, S, p, s_0, F)$  be a Büchi automaton, with  $|S| = n$ . Then we can construct a Büchi automaton  $\bar{A}$  with  $O(16^n)$  states, such that  $L(\bar{A}) = \overline{L(A)}$ .

*Proof:* From the automata  $\tilde{A}_i$  and  $\tilde{A}_j$  (each of size  $m+1 = 4^{n^2} + 1$ ), it is easy to build a Büchi automaton for  $Y_{ij}$  with  $2m+1$  states. Then, by Lemma 2.6, we only need to take the union of the automata for the languages  $Y_{ij}$  such that  $Y_{ij} \cap L(A) = \emptyset$ . Thus  $\bar{A}$  will be a union of at most  $m^2$  Büchi automata, each with at most  $2m+1$  states. The resulting automaton will thus have as many as  $m^2(2m+1) = O(64^{n^2})$  states. However, this construction is rather wasteful. Indeed, it contain as many as  $m^2$  copies of the automata  $\tilde{A}_i$ 's. A more careful construction can use only  $m+1$  copies of these automata. So, we can construct an  $\bar{A}$  that has at most  $O(16^{n^2})$  states.  $\square$

Note that the above proof is not fully constructive, since we did specify how we can check for each  $Y_{ij}$  whether  $Y_{ij} \cap L(A) = \emptyset$  or not. We will address this issue in the next section.

### 2.3. Decision Problems

Two problems that we will have to solve for Büchi automata are the non-emptiness and the non-emptiness of complement problems; that is, given a Büchi automaton, determine whether there is some word it accepts (the non-emptiness problem), and whether there is some word it does not accept (the non-emptiness of complement problem). To solve the non-emptiness problem, we prove the following lemma in the full paper.

*Lemma 2.7:* A Büchi automaton accepts some word iff there is a designated state of the automaton that is reachable from the initial state and is reachable from itself.  $\square$

This will enable us to prove the following:

*Theorem 2.8:* The non-emptiness problem for Büchi automata is logspace complete for NLOGSPACE.  $\square$

We now turn to the non-emptiness of complement problem for Büchi automata. Given a Büchi automaton  $A$ , the obvious way to solve this problem is to construct the automaton  $\bar{A}$  and then use the algorithm for non-emptiness on  $\bar{A}$ . This gives an algorithm that uses exponential time and space as  $\bar{A}$  is of size exponential in the size of  $A$ . However, as the fact that the non-emptiness problem is in NLOGSPACE indicates, it is possible to solve the non-emptiness of complement problem using only polynomial space. The argument is that it is not necessary to first build the whole automaton  $\bar{A}$  before applying the algorithm for non-emptiness. In the rest of this paper, we will have to deal several times with the same type of construction: given an instance of a problem, construct a Büchi automaton which is exponentially big in the size of the problem, then determine if the Büchi automaton accepts some word. Each time, we will be able to show that the problem can be solved using only polynomial space. To avoid repeating the same argument several times, we prove the following lemma in the full paper.

*Lemma 2.9:* Given a problem  $P$  and a Büchi automaton  $A$  which can be constructed from  $P$ , if

- 1) the size of the states of  $A$  is polynomial in the size of  $P$ ,
- 2) the initial state of  $A$  can be constructed in space polynomial in the size of  $P$ ,
- 3) it can be checked if a state is designated in space polynomial in the size of  $P$ , and
- 4) each transitions of  $A$  can be constructed in space polynomial in the size of  $P$  (i.e., given a state of  $A$ , one can construct each transition from that state in polynomial space),

then determining if  $A$  accepts some word can be done in space polynomial in the size of  $P$ .  $\square$

So, to show that the non-emptiness of complement problem for Büchi automata is in PSPACE, we only need to establish that the complement of a Büchi automaton satisfies the conditions of Lemma 2.9. To do this, the only non-obvious step is to show that the correct languages  $Y_{ij}$  can be selected in polynomial space. We thus show the following:

*Lemma 2.10:* The problem of determining if  $Y_{ij} \cap L(A) \neq \emptyset$  is in PSPACE for any of the languages  $Y_{ij}$ .

*Proof:* This can be done by constructing a Büchi automaton  $A_{ij}$  that accepts the language  $L(A) \cap Y_{ij}$  and checking whether it is empty or not. The automaton  $A_{ij}$  can be constructed with at most  $O(n4^n)$  states (cf. [VW84]). Furthermore,  $A_{ij}$  satisfies the conditions of Lemma 2.9. So we can check its emptiness in polynomial space.  $\square$

From Theorem 2.6 and Lemmas 2.9 and 2.10, we can then prove:

*Theorem 2.11:* The non-emptiness of complement problem for Büchi automata is logspace complete in PSPACE.  $\square$

We note that our technique yields a polynomial space upper bound for the equivalence problem of Büchi automata. The previously known algorithm for this problem runs in exponential time and space [Al84].

### 3. Extended Temporal Logic

In [Wo83], an extension of temporal logic that incorporates nondeterministic finite automata as connectives was introduced. In [WVS83], three different versions of this extension were defined and studied further. The difference between the three versions is the type of acceptance conditions used for the finite automata defining the connectives. The three types of acceptance are *finite acceptance* (some prefix is accepted by the standard notion of acceptance for finite words), *looping acceptance* (the automaton has some

infinite run over the word) and *repeating acceptance* (the automaton has a Büchi acceptance condition). These acceptance conditions give rise to three logics:  $ETL_f$ ,  $ETL_i$ , and  $ETL_r$ , correspondingly.

These logics all have the same expressive power. Nevertheless, while there is a linear translation from  $ETL_f$  and  $ETL_i$  to  $ETL_r$ , the best known translation from  $ETL_r$  to  $ETL_f$  or  $ETL_i$  is triply exponential. This suggests that  $ETL_r$  is much more succinct than  $ETL_f$  and  $ETL_i$ . Moreover, while the decision problem for  $ETL_f$  and  $ETL_i$  is PSPACE-complete, the decision procedure for  $ETL_r$  presented in [WVS83] required exponential space. Nevertheless, using our new results on Büchi automata, we will now show that the decision problem for  $ETL_r$  is also in PSPACE, hence it is PSPACE-complete. Note, however, that the decision procedure for  $ETL_f$  and  $ETL_i$  requires space  $O(n^2)$ , while the decision procedure for  $ETL_r$  requires space  $O(n^4)$ .

### 3.1. Definition of the Logic

Formulas of  $ETL_r$  are built from a set of atomic propositions  $P$  by means of

- Boolean connectives, and
- automata connectives: every Büchi automaton  $A = (\Sigma, S, \rho, s_0, F)$ , where  $\Sigma$  is the input alphabet  $\{a_1, \dots, a_k\}$ ,  $S$  is the set of states,  $\rho: S \times S \rightarrow 2^S$  is the transition function,  $s_0 \in S$  is the initial state, and  $F \subseteq S$  is a set of designated states, is considered as an  $n$ -ary temporal connective. That is, if  $f_1, \dots, f_i$  are formulas, then so is  $A(f_1, \dots, f_i)$ .

A structure for our logic is an infinite sequence of truth assignments, i.e., a function  $\pi: \mathbb{N} \rightarrow 2^P$  that assigns truth values to the atomic propositions in each state. We use  $\pi^i$  to denote the  $i$ -th "tail" of  $\pi$ , i.e.,  $\pi^i(k) = \pi(k+i)$ . We now define satisfaction of formulas by induction. Satisfaction of a formula  $f$  by a structure  $\pi$  is denoted  $\pi \models f$ .

- for an atomic proposition  $p$ ,  $\pi \models p$  iff  $p \in \pi(0)$ .
- $\pi \models f_1 \wedge f_2$  iff  $\pi \models f_1$  and  $\pi \models f_2$ .
- $\pi \models \neg f$  iff not  $\pi \models f$ .
- $\pi \models A(f_1, \dots, f_i)$ , where  $A = (\Sigma, S, \rho, s_0, F)$ , iff there exists an infinite word  $w = w_0 w_1 \dots$  over  $\Sigma$ , accepted by  $A$ , such that for all  $j \geq 0$ , if  $w_j$  is  $a_k$ , then  $\pi^j \models f_k$ .

### 3.2. Decision Procedure for $ETL_r$

To give the PSPACE decision procedure for  $ETL_r$ , we first need to introduce the notion of the closure of an  $ETL_r$  formula  $f$ , denoted  $cl(f)$ . Given an automaton  $A = (\Sigma, S, \rho, s_0, F)$ , for each  $s \in S$  we define  $A_s$  to be the automaton  $(\Sigma, S, \rho, s, F)$ . The closure is then defined as follows:

- $f \in cl(f)$
- $f_1 \wedge f_2 \in cl(f) \rightarrow f_1, f_2 \in cl(f)$
- $\neg f_1 \in cl(f) \rightarrow f_1 \in cl(f)$
- $f_1 \in cl(f)$  not of the form  $\neg f_2 \rightarrow \neg f_1 \in cl(f)$
- $A(f_1, \dots, f_i) \in cl(f) \rightarrow f_1, \dots, f_i \in cl(f)$
- $A(f_1, \dots, f_i) \in cl(f) \rightarrow A_s(f_1, \dots, f_i) \in cl(f)$ , for all  $s \in S$ .

If we define the size of an automaton connective to be equal to the number of states of the automaton, then for an  $ETL_r$  formula  $f$ , the size of  $cl(f)$  can easily be seen to be at most  $2n$  where  $n$  is the length of  $f$ .

To establish a decision procedure for  $ETL_r$ , we reduce the satisfiability problem to the emptiness problem for Büchi automata over the alphabet  $2^{cl(f)}$ . To this end we extend the function  $\pi: \mathbb{N} \rightarrow 2^P$  to a function  $\psi: \mathbb{N} \rightarrow 2^{cl(f)}$  in a natural way: for every  $i \in \mathbb{N}$  and every formula  $g \in cl(f)$ , we have that  $g \in \psi(i)$  iff  $\pi^i$  satisfies  $g$ . Sequences that correspond to models satisfy some special properties.

A *Hintikka sequence* for an  $ETL_r$  formula  $f$  is a function  $\psi: \mathbb{N} \rightarrow 2^{cl(f)}$  that satisfies the following conditions:

- 1)  $f \in \psi(0)$
- and, for all elements  $i \in \mathbb{N}$
- 2)  $g \in \psi(i)$  iff  $\neg g \notin \psi(i)$ ,
  - 3)  $g_1 \wedge g_2 \in \psi(i)$  iff  $g_1 \in \psi(i)$  and  $g_2 \in \psi(i)$ ,
  - 4) If  $A(f_1, \dots, f_i) \in \psi(i)$ , where  $A = (\Sigma, S, \rho, s_0, F)$ , then there exists an infinite word  $w = w_0 w_1 \dots$  over  $\Sigma$ , accepted by  $A$ , such that for all  $j \geq 0$ , if  $w_j$  is  $a_k$ , then  $f_k \in \psi(i+j)$ .

- 5) If  $\neg A(f_1, \dots, f_i) \in \psi(i)$ , then there is no infinite word  $w = w_0 w_1 \dots$  over  $\Sigma$ , accepted by  $A$ , such that for all  $j \geq 0$ , if  $w_j$  is  $a_k$ , then  $f_k \in \psi(i+j)$ .

**Lemma 3.1:** A  $ETL_r$  formula  $f$  has a model iff it has a Hintikka sequence

*Proof:* Given in the full paper.  $\square$

The next step in obtaining a decision procedure for  $ETL_r$  is to construct a Büchi automaton that accepts exactly the Hintikka sequences for a formula. To do this, we will actually build three automata. The *local automaton*,  $A_L$ , will check Hintikka conditions 1-3, the *positive automaton*,  $A_P$ , will check Hintikka condition 4 and the *negative automaton*,  $A_N$ , will check Hintikka condition 5. The local automaton is straightforward to construct and we leave it to the reader.

#### The Positive Automaton

The positive automaton is actually the result of taking the intersection of a collection of automata, one for each formula of the form  $A(f_1, \dots, f_i)$  in  $cl(f)$ . We will now describe how to build each of these automata. We need to construct an automaton defined over  $2^{cl(f)}$  that will run  $A$  for each  $j$  such that  $A(f_1, \dots, f_i) \in \psi(j)$ . We will build this automaton from  $A$  in several steps.

Let  $A = (\Sigma, S, p, s_0, F)$ , where  $\Sigma = \{a_1, \dots, a_i\}$ . First, we will transform  $A$  into an automaton  $A^0$  over  $2^{cl(f)}$ . The states of  $A^0$  are the same as those of  $A$  and we have that  $s' \in \rho^0(a, s)$  iff for some  $j$ ,  $s' \in \rho(a_j, s)$  and  $f_j \in a$ .

We now need to transform the automaton  $A^0$  so that it checks the sequence each time  $A(f_1, \dots, f_i)$  appears. To do this, we will use a construction similar to the "flag construction" described in [Ch74, Ra70]. Details will be given in the full paper.

#### The Negative Automaton

The most straightforward way to deal with this case is to build, for each formula of the form  $\neg A(f_1, \dots, f_m)$ , the complement of the automaton  $A$  and then apply the flag construction as in the preceding case. This, however, would lead to a doubly exponential blow-up and it is possible to be more efficient. We do that by first building an automaton that tries to find an occurrence of  $\neg A_P(f_1, \dots, f_i)$  that does not satisfy condition 5, and then taking the complement of this automaton.

We start by constructing the automaton  $A^0$  exactly as in the previous case. We then build an automaton  $A^1$ . The automaton  $A^1$  has the same states as the automaton  $A^0$  plus a dormant state 0. Its transition function is the one of  $A^0$  extended, for all  $a \in 2^{cl(f)}$ , by  $0 \in \rho^1(a, 0)$ ; and, for all  $a$  such that  $\neg A(f_1, \dots, f_n) \in a$ , it is extended by  $s \in \rho^1(a, 0)$  for each  $s$  such that  $s \in \rho^0(a, s_0)$ . Acceptance is defined exactly as for  $A$ . The automaton we have built stays in the dormant state until it sees an element containing  $\neg A(f_1, \dots, f_i)$  and then either stays in the dormant state or starts running exactly as  $A^0$ . The sequences it accepts are thus those in which  $\neg A(f_1, \dots, f_n)$  appears at some point and that satisfy  $A(f_1, \dots, f_i)$  from that point. This is exactly the complement of the set of sequences we are checking for. We now take  $A^2$  to be the union of the automata  $A^1$  for each formula of the form  $\neg A(f_1, \dots, f_i)$  in  $cl(f)$ .  $A^2$  has at most  $n+1$  states, where  $n$  is the length of  $f$ . Now  $A_N$  is taken to be the complement of  $A^2$ , using the construction we described in Section 2.  $A_N$  has  $O(16^{n^2+2n})$  states.

We now have:

**Proposition 3.2:** Let  $f$  be an  $ETL_r$  formula. Then, one can construct a Büchi automaton of size exponential in the length of  $f$  such that a sequence  $\psi: \mathbb{N} \rightarrow 2^{cl(f)}$  is accepted by that automaton iff it is a Hintikka sequence for  $f$ .

*Proof:* The automaton that corresponds to the intersection of the local, positive, and negative automata for  $f$  is the required automaton. This automaton has  $O(c^{n^2+n \log n})$  states, for some constant  $c$ .  $\square$

**Proposition 3.3:** The satisfiability problem for  $ETL_r$  is log space complete in PSPACE.

*Proof:* The hardness result follows easily from the hardness results in [SC82]. To prove that the problem is in PSPACE, it is sufficient to observe that the automata,  $A_L$ ,  $A_P$ , and  $A_N$  satisfy the conditions of Lemma 2.9. Thus the automaton corresponding to their intersection also satisfies the conditions of Lemma 2.9, and the problem of determining if this automaton accepts some word, which is equivalent by Proposition 3.1 to determining if the formula is satisfiable is in PSPACE.  $\square$

The above proof shows that the decision procedure for  $ETL_r$  requires nondeterministic space  $O(n^2)$ , and consequently deterministic space  $O(n^4)$ . In contrast, the decision procedures for  $ETL_r$  and  $ETL$  requires nondeterministic space  $O(n)$ , and consequently deterministic space  $O(n^2)$ .

#### 4. Quantified Proposition Temporal Logic

In the previous section, we proved a result about one possible extension of temporal logic. There are other ways to extend temporal logic, one of which is to introduce quantification over propositions. This extension, quantified propositional temporal logic (QPTL), was described in [Sis83]. It turns out that it has exactly the same expressive power as the extended temporal logic we studied in section 3 [Wo82, WVS83]. However, as we will see, from a complexity point of view it has very different properties.

Formulas of QPTL are built from a set of atomic propositions  $P$  using:

- boolean connectives
- the temporal operators  $X$  (next) and  $F$  (eventually). We will also use  $G$  as an abbreviation for  $\neg F \neg$ .
- Quantification over propositions (i.e., if  $f(p)$  is a formula, then so is  $\exists p f(p)$ ). We will also use  $\forall$  as an abbreviation for  $\neg \exists \neg$ .

We will say that a QPTL formula is in *normal form* if it can be written as:

$$Q_1 p_1 Q_2 p_2 \cdots Q_k p_k f$$

where each  $Q_i$  is either  $\forall$  or  $\exists$  and  $f$  is a quantifier-free formula. (Every QPTL formula is equivalent to a formula in normal form.) If  $Q_1$  is  $\exists$  and there are  $k$  alternations of quantifiers, we say that the formula is in the set  $\Sigma_k^{QPTL}$ .

QPTL formulas are interpreted over infinite sequence of truth assignments, i.e., functions  $\pi: \mathbb{N} \rightarrow 2^P$  that assigns truth values to the atomic propositions in each state. We use  $\pi^i$  to denote the  $i$ -th "tail" of  $\pi$ , i.e.,  $\pi^i(k) = \pi(k+i)$ . We now define satisfaction of formulas inductively.

- for an atomic proposition  $p$ ,  $\pi \models p$  iff  $p \in \pi(0)$ .
- $\pi \models f_1 \wedge f_2$  iff  $\pi \models f_1$  and  $\pi \models f_2$ .
- $\pi \models \neg f$  iff not  $\pi \models f$ .
- $\pi \models Xf$  iff  $\pi^1 \models f$
- $\pi \models Ff$  iff there is a  $i \geq 0$  such that  $\pi^i \models f$
- $\pi \models \exists p f$  iff there is some  $\pi'$  that agrees with  $\pi$  except for the proposition  $p$  and such that  $\pi' \models f$

Before stating our complexity results on QPTL, we need one definition. Let us define  $g(k, n)$  as follows:

$$g(0, n) = n$$

$$g(k+1, n) = 2^{g(k, n)}.$$

(i.e.,  $g(k, n)$  has a stack of  $k$  exponents.)

We prove the following:

**Theorem 4.1:** The satisfiability problem for  $\Sigma_k^{QPTL}$  is complete for  $\text{SPACE}(g(k, p(n)))$ , where  $p(n)$  is a polynomial.  $\square$

We note that this result also holds for *weak* QPTL, in which all predicate are finite, i.e., they are eventually false forever. A result closely related to ours was proven by Robertson [Ro74]. Robertson studied WS1S, the theory of natural numbers of successor with quantification over finite sets, which is equivalent to weak QPTL. He showed that  $\Sigma_k^{WS1S}$  is in  $\text{NTIME}(g(k+1, p(n)))$ , and is logspace hard for  $\text{NTIME}(g(k, p(n)))$ . Thus we improve his results both by closing the gap between lower and upper bounds, and by being able to deal also with quantification over infinite sets. We believe our result to be of general theoretical interest, since QPTL is the first nonelementary logic we know of, where each alternation increases the complexity by exactly one exponential.

##### 4.1. Upper Bounds

The proof of our upper bounds, will be based on the construction of a Büchi automaton that accepts exactly the sequences satisfying a  $\Sigma_k^{QPTL}$  formula. More precisely, we prove the following.

**Lemma 4.2:** Given a  $\Sigma_k^{QPTL}$  formula  $f$  of size  $n$ , one can construct a Büchi automaton of size  $g(k+1, p(n))$  that accepts exactly the sequences satisfying  $f$ .

*Proof:* The proof proceeds by induction.

*Base case* ( $k=0$ ): In this case one deals with quantifier free formulas and the construction given in [WVS83] can be used.

*Inductive Step*: We have to establish the result for formulas in  $\Sigma_k^{QPTL}$  knowing the result for formulas in  $\Sigma_{k-1}^{QPTL}$ . First, notice that a formula in  $\Sigma_k^{QPTL}$  can be written as

$$\exists \neg f_1$$

where  $f_1$  is a formula in  $\Sigma_{k-1}^{QPTL}$ . Now, if we can build an automaton of size  $g(k, p(n))$  for  $f_1$ , we can construct an automaton for  $\exists \neg f_1$  by complementing that automaton and projecting it on a smaller alphabet to eliminate the quantified propositions. If we use the complementation algorithm for Büchi automata described in Section 2, the result will be an automaton of size  $g(k+1, p(n))$ .  $\square$

We can now establish our upper bounds.

**Theorem 4.3:** Satisfiability for formulas of  $\Sigma_k^{QPTL}$  is in  $\text{SPACE}(g(k, p(n)))$ .

*Proof*: Let us first consider the case  $k=0$ . We have to show that satisfiability for quantifier free temporal logic formulas is in PSPACE. This was done in [SC82]. Also, this is an immediate consequence of the fact that the Büchi automaton we constructed for this case satisfies all the conditions of Lemma 2.9.

In the case  $k \geq 1$ , we have a formula  $f$  of the form  $\exists \neg f_1$  where  $f_1 \in \Sigma_{k-1}^{QPTL}$ . By Lemma 4.2, we know that we can construct an automaton for  $f_1$  using space  $g(k, p(n))$ . Now, to check if the formula  $f$  is satisfiable, it is sufficient to check that there is some word not accepted by the automaton for  $f_1$ . By Lemma 2.11, this can be done in space polynomial in the size of the automaton for  $f_1$  and hence in  $\text{SPACE}(g(k, p'(n)))$  for some polynomial  $p'$ .  $\square$

## 4.2. Lower Bounds

We will now show the lower bound for  $\Sigma_k^{QPTL}$ . To do this, we first need to show how to construct a formula  $Q_{k,m}(p, q)$  such that  $p$  and  $q$  are true exactly once and are separated by a distance greater than  $g(k, m)$ . More precisely, we prove the following lemma.

**Lemma 4.4:** Given  $k > 0$  and  $m > 0$ , one can construct in space  $\log(m)$  a formula  $\phi_{k,m}(p, q) \in \Sigma_{k-1}^{QPTL}$  such that if  $\pi \models \phi_{k,m}(p, q)$  then

- 1)  $p$  and  $q$  are each true at exactly one point.
- 2) If  $\pi^i \models p$  and  $\pi^j \models q$  then  $j = i + N_{k,m}$  for some number  $N_{k,m}$  such that  $N_{k,m} \geq g(k, m)$ .

*Proof*:

The proof will proceed by induction. We show how to construct  $\phi_{1,m}$  and then show how to construct  $\phi_{k+1,m}$  given  $\phi_{k,m}$ . Before proceeding further, we need to define the notion of a *proper sequence of numbers*. Let  $c_0, c_1, \dots, c_{l-1}$  be a sequence of binary numbers each of size  $s$ . We let  $v(c_i)$  denote the integer value of the number  $c_i$ , and  $c_{ij}$  denote the  $j^{\text{th}}$  bit in the number  $c_i$  (the 0th bit is the least significant bit). We say that the above sequence is a *proper sequence of numbers* iff  $v(c_0) = 0$ ,  $v(c_{i+1}) = v(c_i) + 1$  for  $0 \leq i < l-1$ , and  $v(c_{l-1}) = 2^s - 1$ . Clearly for a proper sequence  $l = 2^s$ . It is easy to write the relation between the bits in successive numbers. For  $0 \leq i < l-1$  and  $0 \leq j < s$  we have

$$c_{(i+1)j} = c_{ij} \text{ iff } \exists r < j \text{ such that } c_{ir} = 0$$

We can now show how to construct  $\phi_{1,m}(p, q)$ . For this we use  $m$  existentially quantified propositions  $q_0, q_1, \dots, q_{m-1}$ . At each point in a sequence, the truth values of these propositions define a binary number. The formula  $\phi_{1,m}$  asserts that the sequence of these numbers starting at the point  $i$  where  $p$  is true form a proper sequence of numbers. The formula will also assert that there is exactly one point just before the point where  $q$  is true where all the propositions  $q_0, \dots, q_{m-1}$  are true. It is fairly straightforward to construct these formulas. Thus,  $\phi_{1,m}$  can be constructed and it will be of length  $O(m^2)$ . It satisfies condition 2 of the lemma if we take  $N_{1,m} = 2^m$ .

We now show how to construct  $\phi_{k+1,m}$  given  $\phi_{k,m}$  for  $k \geq 1$ . Intuitively, the formula  $\phi_{k+1,m}(p, q)$  will state that there is a proper sequence of numbers each of length  $N_{k,m}$  between the points  $i, j$  where  $p$  and  $q$  are true respectively. Each number will occupy  $N_{k,m}$  positions in the sequence and its value will be given by the truth values of a proposition  $b$  on those positions. The formula will also use a proposition  $r$  which will be true at each position corresponding to the beginning of a number. The formula  $\phi_{k+1,m}(p, q)$  will be the conjunction of formulas stating the following conditions:

1. The proposition  $p$  holds at exactly one point, say  $i$ , and  $q$  holds at exactly one point say  $j$ . The proposition  $r$  is true at  $i$  and  $j$ , and is false at all points before  $i$  and after  $j$ .

2. The successive points where  $r$  is true, are separated by a distance  $N_{k,m}$ . We can satisfy this by requiring that for all points  $k, l$ , if  $k$  and  $l$  are separated by a distance  $N_{k,m}$  then there is exactly one instance between  $k$  and  $l$  where  $r$  is true.

By this condition we can consider the truth values of  $b$ , between successive instances where  $r$  is true, to be a binary number of size  $N_{k,m}$ . We consider the right most bit of a number to be its least significant bit.

3. The proposition  $b$  is false throughout the subsequence between the first and second positions where  $r$  is true. This asserts that the value of the first number is 0.

4. The value of each succeeding number is equal to the value of the preceding number incremented by 1. This can be expressed by giving the relation between the values of the bits in corresponding positions of successive numbers. Such positions are those between  $i$  and  $j$ , which are separated by a distance  $N_{k,m}$ .

5. In the last number, that is the one before  $j$ ,  $b$  is true at all points in the number.

6. In all numbers other than the last number there is at least one position where  $b$  is false. This condition guarantees that 5 does not hold in any other number.

Let  $f'$  be the conjunction of the formulas expressing conditions 1-6 and let  $f$  be the result of converting  $f'$  into standard form. In the full paper, we will show that  $\phi_{k+1,m}(p, q) = \exists b \forall p_i p_i(f)$ . Also, we will show that since  $\phi_{k,m} \in \Sigma_k^{QPTL}$ ,  $f \in \Pi_{k-1}^{QPTL}$ . Thus,  $\phi_{k+1,m} \in \Sigma_k^{QPTL}$  and  $\phi_{k+1,m}$  satisfies the second condition of the lemma for an  $N_{k,m}$  satisfying

$$N_{k,m} = N_{k-1,m} \cdot 2^{(2^{N_{k-1,m}})} \text{ for } k \geq 2$$

Given that the construction for  $\phi_{1,m}$  gave us  $N_{1,m} = 2^m$ , we clearly have that  $N_{k,m} \geq g(k, m)$ . Finally, it is easily seen that the length of  $\phi_{k,m}$  is  $O(m^2)$ , and it can be obtained in  $SPACE(\log m)$ .  $\square$

We are now ready to prove our lower bounds.

**Theorem 4.5:** Every language in  $SPACE(g(k, p(n)))$  is log space reducible to the satisfiability problem for  $\Sigma_k^{QPTL}$ .

*Proof:* Let  $M = (Q, \Sigma, p, V_A, V_I)$  be a one tape deterministic turning machine that is  $g(k, p(n))$  space bounded. The elements in  $M$  are the set of states, the alphabet, the next move function, the accepting state and the initial state respectively. Let  $ID_0, ID_1, \dots$  be a sequence of  $ID$ s that describe the computation of  $M$  on some input of length  $n$ . Let  $m = p(n)$ . Without loss of generality we assume that each  $ID$  is of length  $N_{k,m}$ . If  $N_{k,m} \geq g(k, m)$  then  $M$  uses only the initial  $g(k, m)$  cells in each  $ID$ . Using a formula  $f$  of  $QPTL$  we express the computation of  $M$  on input  $w$ .

We use the propositions  $P_\sigma$  for each  $\sigma \in (Q \times \Sigma) \cup \Sigma$ , where the elements in  $Q \times \Sigma$  are the composite symbols. We use a proposition  $B$  which marks the beginning of  $ID$ s. The sequence between successive instances where  $B$  holds defines an  $ID$ . We briefly describe how we can obtain a formula  $f$  that expresses the computation of  $M$ . The formula  $f$  will be the conjunction of the formulas expressing the following conditions.

1. All the  $ID$ s are of length  $N_{k,m}$ .
2. Each  $ID$  is a valid  $ID$ .
3. Each successive  $ID$  is obtained from the previous one by one move of  $M$ , i.e. the contents of a cell in an  $ID$  depend on the contents of this cell and its neighbor's contents in the preceding  $ID$ .
4.  $ID_0$  is an initial  $ID$ , and eventually a final  $ID$  appears.

Given that  $\phi_{k,m} \in \Sigma_k^{QPTL}$ , we will show in the full paper that each of the conditions 1-4 can be expressed by a formula in  $\Pi_{k-1}^{QPTL}$ . Thus the conjunction  $f$  of these formulas will also be in  $\Pi_{k-1}^{QPTL}$  and the formula  $g$  obtained by introducing an existential quantification over each free variable in  $f$  is in  $\Sigma_k^{QPTL}$ . Moreover,  $g$  is a satisfiable formula iff  $w$  is accepted by  $M$  and it can be obtained using space  $O(\log n)$ .  $\square$

## 5. References

- [Al84] H. Alaiwan, "Equivalence of Infinite Behavior of Finite Automata", *Theoretical Computer Science* 31(1984), pp. 297-306.
- [Bu62] J. R. Büchi, "On a Decision Method in Restricted Second Order Arithmetic", *Proc. Internat. Congr. Logic, Method and Philos. Sci. 1960*, Stanford University Press, 1962, pp. 1-12.
- [Bu73] J. R. Büchi, "The Monadic Theory of  $\omega_1$ ", In *Decidable Theories II*, Lecture Notes in Mathematics, v. 328, Springer-Verlag, 1973, pp. 1-127.

- [Ch74] Y. Choueka, "Theories of Automata on  $\omega$ -Tapes: A Simplified Approach", *J. Computer and System Sciences*, 8 (1974), pp. 117-141.
- [Ei74] S. Eilenberg, *Automata, Languages and Machines*, vol. A, Academic Press, New York, 1974.
- [HKP82] D. Harel, D. Kozen, R. Parikh, "Process Logic: Expressiveness, Decidability, Completeness", *Journal of Computer and System Science* 25, 2 (1982), pp. 144-170.
- [McN66] R. McNaughton, "Testing and Generating Infinite Sequences by a Finite Automaton", *Information and Control* 9 (1966), pp. 521-530
- [Me75] A. R. Meyer, "Weak Monadic Second Order Theory of Successor is not Elementary Recursive", *Proc. Logic Colloquium*, Lecture Notes in Mathematics, v. 453, Springer-Verlag, 1975, pp. 132-154.
- [MS72] A. R. Meyer, L. J. Stockmeyer, "The Equivalence Problem for Regular Expressions with Squaring Requires Exponential Time", *Proc. 13th IEEE Symp. on Switching and Automata Theory*, Long Beach, CA, 1972, pp. 125-129.
- [Ni80] H. Nishimura, "Descriptively Complete Process Logic", *Acta Informatica*, 14 (1980), pp. 359-369.
- [Pn81] A. Pnueli, "The Temporal Logic of Concurrent Programs", *Theoretical Computer Science* 13(1981), pp. 45-60.
- [Ra69] M.O. Rabin, "Decidability of Second Order Theories and Automata on Infinite Trees", *Trans. AMS*, 141(1969), pp. 1-35.
- [Ra70] M.O. Rabin, "Weakly Definable Relations and Special Automata", *Proc. Symp. Math. Logic and Foundations of Set Theory* (Y. Bar-Hillel, ed.), North-Holland, 1970, pp. 1-23.
- [Ro74] E. L. Robertson, "Structure of Complexity in the Weak Monadic Second-Order Theory of the Natural Numbers", *Proc. 6th ACM Symp. on Theory of Computing*, Seattle, 1974, pp. 161-171.
- [RS59] M. O. Rabin, D. Scott, "Finite Automata and their Decision Problems", *IBM J. Res. & Dev.*, 3(2), 1959, pp 114-125.
- [SC82] A.P. Sistla, E.M. Clarke, "The Complexity of Propositional Linear Time Logics", *Proc. 14th ACM Symp. on Theory of Computing*, San Francisco, 1982, pp. 159-168, to appear in *J. ACM*.
- [Sie70] D. Siefkes, *Decidable Theories I - Büchi's Monadic Second-Order Successor Arithmetics*, Lecture Notes in Mathematics, v. 120, Springer-Verlag, 1970.
- [Sis83] A. P. Sistla, *Theoretical Issues in The Design and Verification of Distributed Systems*, Ph.D. Thesis, Harvard University, 1983.
- [TB73] B. A. Trakhtenbrot, Y. M. Barzdin, *Finite Automata Behavior and Synthesis*, North Holland, Amsterdam, 1973.
- [Va85] M.Y. Vardi, "On deterministic  $\omega$ -automata", to appear.
- [VS85] M.Y. Vardi, L. Stockmeyer, "Improved Upper and Lower Bounds for Modal Logics of Programs", *Proc. 17 ACM Symp. on Theory of Computing*, Providence, May 1985.
- [VW83] M. Y. Vardi, P. Wolper, "Yet Another Process Logic", in *Logics of Programs*, Springer-Verlag Lecture Notes in Computer Science, vol. 164, Berlin, 1983, pp. 501-512.
- [VW84] M. Y. Vardi, P. Wolper, "Automata Theoretic Techniques for Modal Logics of Programs", IBM Research Report, October 1984. A preliminary version appeared in *Proc. ACM Symp. on Theory of Computing*, Wahington, April 1984, pp. 446-456.
- [WVS83] P. Wolper, M. Y. Vardi, A. P. Sistla, "Reasoning about Infinite Computation Paths", *Proc. 24th IEEE Symp. on Foundations of Computer Science*, Tucson, 1983, pp. 185-194.
- [Wo82] P. Wolper, "Synthesis of Communicating Processes from Temporal Logic Specifications", Ph. D. Thesis, Stanford University, 1982.
- [Wo83] P. Wolper, "Temporal Logic Can Be More Expressive", *Information and Control*, 56(1983), pp. 72-99.